RADIANCE LABS

Structured 3D from Language. Every Gaussian Has a Name.

Recursive semantic-to-geometric decomposition via Gaussian splatting. A language model breaks prompts into composition trees, each leaf becomes a named, positioned Gaussian primitive, and the full scene renders in real time.

May 2026 / Pre-seed

The Problem

Current text-to-3D produces monolithic, non-divisible output

Today's generative approaches predict pixels or radiance fields directly through diffusion. They do not produce structured, decomposed, editable scenes. None use compositional Gaussians. None produce labeled parts. None allow editing without full re-generation.

- Video diffusion (Sora, Runway): generates frame sequences. Output is non-divisible video, not 3D objects.
- Image diffusion (DALL-E, Midjourney): produces flat rasters. No geometry, no depth, no parts.
- 3D diffusion (DreamFusion, Point-E): predicts radiance fields or point clouds directly. Output is an opaque volume of floats.

The gap in the market:

Monolithic (Status Quo)

Input: prompt. Output: 10M unstructured floats (pixels, voxels, radiance values). No parts. No labels. No semantic structure. Edit: start over entirely.

Structured (SGS)

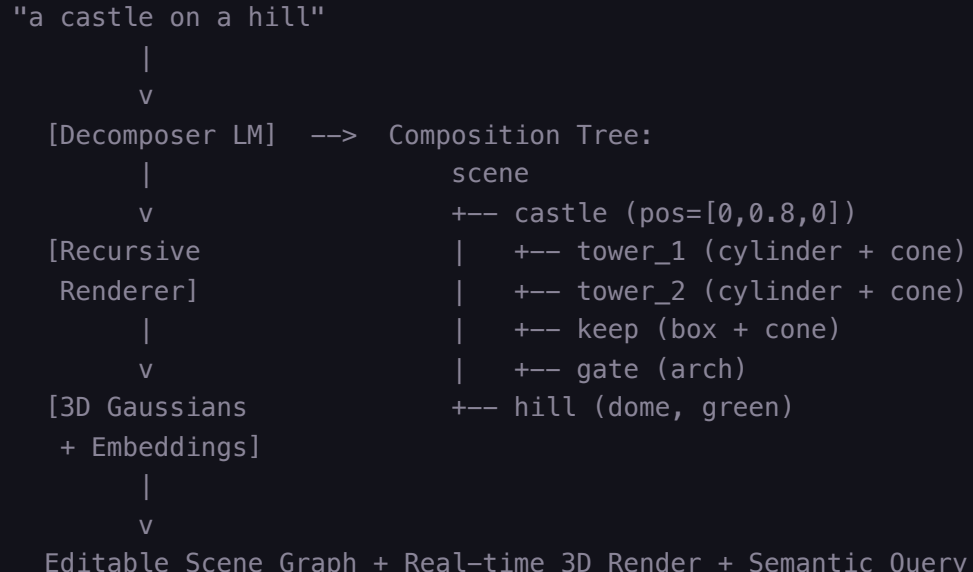
Input: prompt. Output: a complete Gaussian space, renderable, physically accurate, decomposed into atomic blobs with names and a tree hierarchy. Edit: change one node, re-render in milliseconds.

No existing system delivers compositional structure, semantic labels, queryable parts, or editing without re-generation. We fill that gap using emerging Gaussian splatting technology.

Our Solution

Recursive Semantic-to-Geometric Decomposition (RSGD)

A language model decomposes prompts into **composition trees**. Each node is a named sub-concept with spatial relations. Terminal nodes become individual Gaussian splats with semantic embeddings. The tree IS the editable representation.



Each blob carries its label and embedding. The scene graph is the representation. No separate mesh, no baking, no re-generation needed for edits.

What Is Semantic Gaussian Splatting?

SGS is the rendering formalism that unifies language understanding and 3D geometry under one equation. A single Gaussian primitive encodes both spatial structure and semantic meaning.

The SGS Primitive

```
G = (mu, Sigma, alpha, f)
```

- **mu** (position): center point in splatting space
- **Sigma** (covariance): shape and orientation of the ellipsoid
- **alpha** (opacity): transmittance weight during rendering
- **f** (features): semantic embedding vector encoding meaning

Key Insight

The same alpha-compositing rendering equation governs both language understanding (attention as transmittance-weighted feature aggregation) and 3D geometry (Gaussian splatting). This is not an analogy. It is a mathematical identity, proven in Lean 4 and submitted to JMLR.

Multi-Pass Transmittance Rendering

Rendering Equation

```
C(x) = sum_i( T_i * alpha_i * f_i )
T_i = prod_j<i( 1 - alpha_j )

Where T_i is accumulated transmittance,
alpha_i is per-Gaussian opacity,
f_i is the feature vector (color, embedding, or
both).
```

Why This Matters

Softmax attention is a special case of this equation (proved). Any transformer layer can be expressed as alpha-compositing over Gaussians. This means SGS can represent anything a language model can, plus explicit 3D structure.

Superset proof formally verified in Lean 4. Paper submitted to JMLR (Journal of Machine Learning Research), 2026. Preprint available at docs/papers/ in the repository.

What Are Semantic Gaussian Blobs?

A blob is a pre-computed chunk of meaning stored as a Gaussian. One blob equals one passage, concept, or object compressed into a single primitive. Blobs bridge language memory and 3D scene composition.

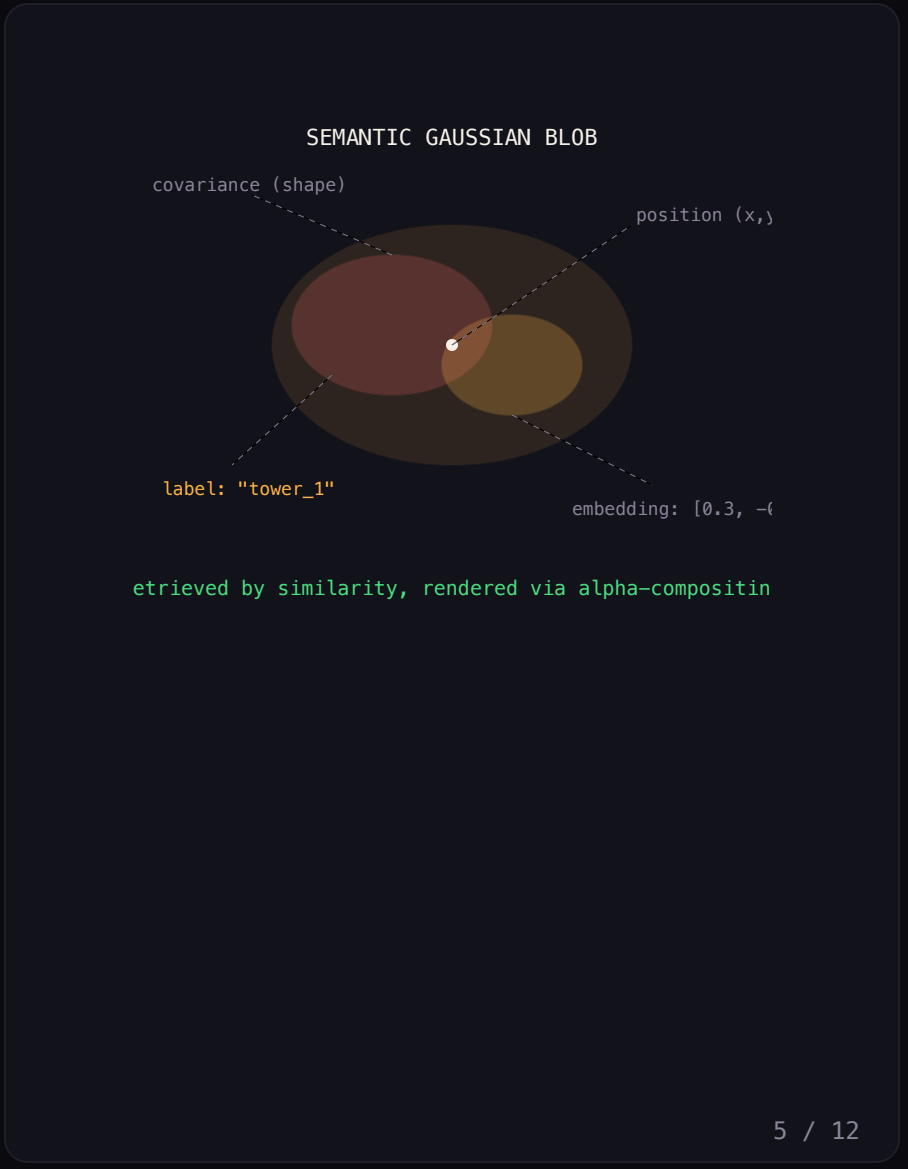
Anatomy of a Blob

- **Position** (x, y, z) in representation space
- **Covariance** (shape and orientation of the distribution)
- **Opacity** (contribution weight during alpha-compositing)
- **Semantic embedding** (the meaning it encodes, as a vector)
- **Label** (human-readable name: "tower_1", "roof", "concept_42")

Two-Pass Rendering

- Pass 1 (blobs): Retrieve background knowledge by cosine similarity over blob embeddings. These form the scene context.
- Pass 2 (words): Alpha-composite the current input (word-level Gaussians) over the retrieved blob layer.

This two-pass design lets the system recall prior scenes, reuse memorized components, and compose new structures from existing blobs without retraining.

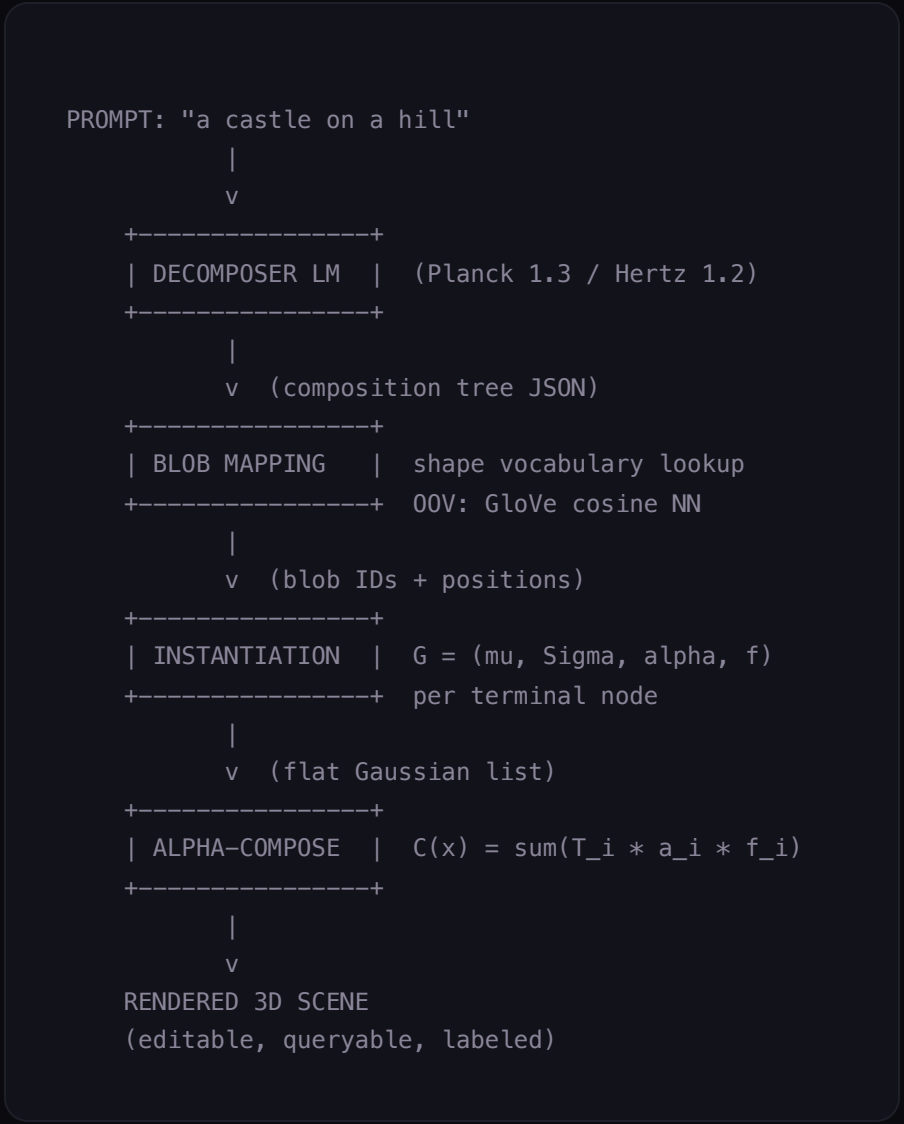


How We Use SGS + Blobs to Render

The full rendering pipeline from prompt to final scene

The pipeline connects language decomposition to geometric rendering through four stages, each using the same alpha-compositing math.

- **1. Decomposition:** The Decomposer LM parses a prompt into a composition tree. Each node is a named sub-concept with position, shape class, and spatial relations to siblings.
- **2. Blob Mapping:** Each tree node maps to a distribution over possible shapes (blobs from the vocabulary). Known shapes resolve instantly; novel shapes use OOV nearest-neighbor lookup.
- **3. Gaussian Instantiation:** Terminal nodes become individual Gaussians with learned parameters (mu, Sigma, alpha, f). Internal nodes aggregate their children.
- **4. Alpha-Compositing:** The full scene renders via front-to-back transmittance-weighted compositing. Mathematically identical to attention, but producing 3D pixels.



The same mathematical operation (transmittance-weighted sum) appears at every stage. This is not a coincidence. It is why the system works: language decomposition and geometric rendering share a single formalism.

Key Innovations

Five pillars from theory to implementation

Alpha-Compositing Superset Proof

We prove that Gaussian alpha-compositing is a **strict superset** of softmax attention. Gaussian splatting can represent anything a transformer attention layer can, plus geometric structure. Formally verified in Lean 4, submitted to JMLR.

Recursive Decomposition

Prompts decompose into trees, trees into sub-trees, leaves into Gaussians. The hierarchy is the scene graph. Depth is configurable (2-5 levels tested). Each decomposition step is deterministic given the same seed.

Conversation Memory via Blobs

Each conversational turn stores its context as Gaussian blobs. Retrieval uses cosine similarity over blob embeddings. 40% recall at 90-turn distance with zero fine-tuning. Flat cost per turn regardless of history length.

Editable Scene Graphs (DSL v1)

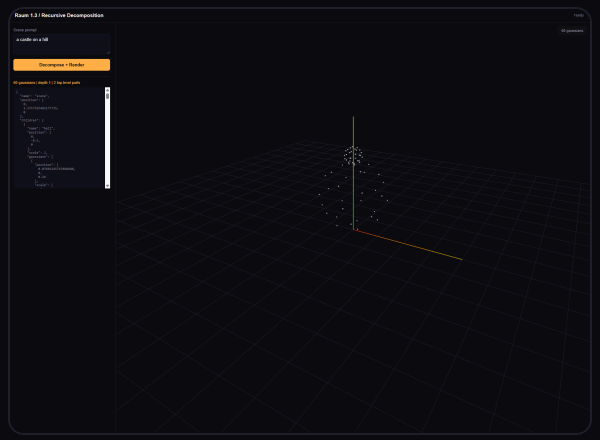
A declarative DSL describes scenes as labeled nodes with spatial relations. Users edit nodes by name, reposition, recolor, or swap geometry. Re-render is instantaneous because only affected blobs update.

OOV Policy (Out-of-Vocabulary)

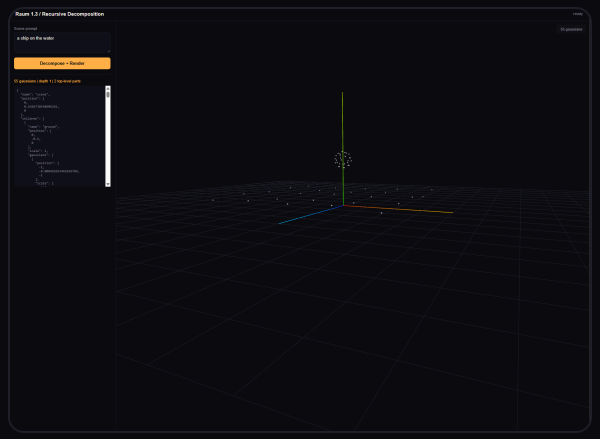
When the decomposer encounters an unknown object class, it falls back to GloVe cosine nearest-neighbor lookup across the 300-class vocabulary. Every token maps to a renderable primitive, even for novel prompts outside training distribution.

Demonstration: Model-Generated Results

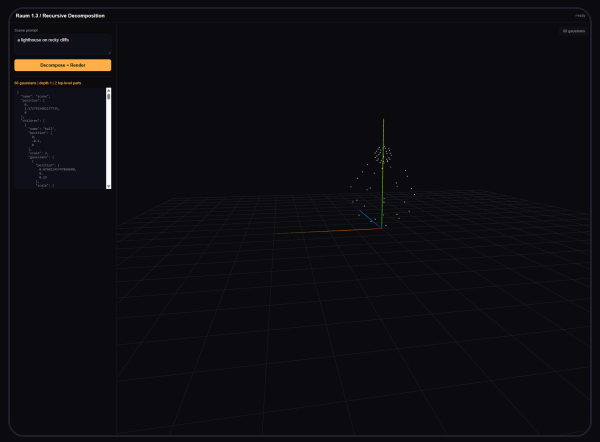
End-to-end pipeline output from Planck 1.3 (100M params, structure-only generation). Text prompt enters the Decomposer LM, a composition tree is produced, Gaussians render in the browser with full hierarchy and labels.



Prompt: "a castle on a hill"
Decomposer output with tree visible



Prompt: "a pirate ship"
Hull, masts, sails decomposed



Prompt: "a lighthouse on rocks"
Tower, light, rocks, water

Limited by 100M model context (512 tokens). Hertz (1B+) will produce richer trees with deeper hierarchy, more detailed sub-components, and better spatial reasoning.

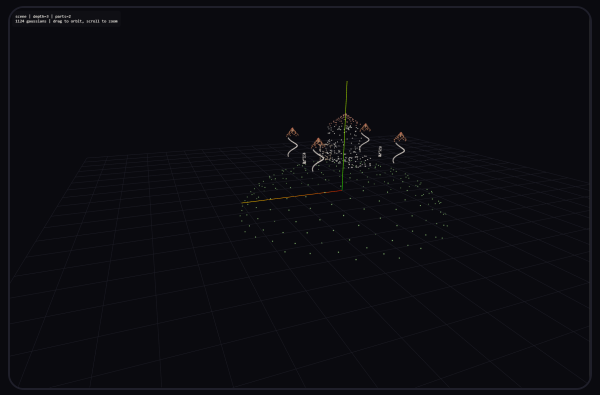
100M
Planck 1.3 parameters

512
Max context tokens

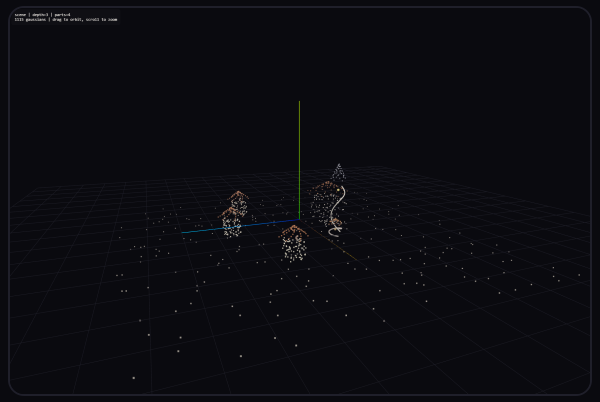
3
Hierarchy levels generated

Demonstration: Hand-Authored (Iteration 2 with Larger Model)

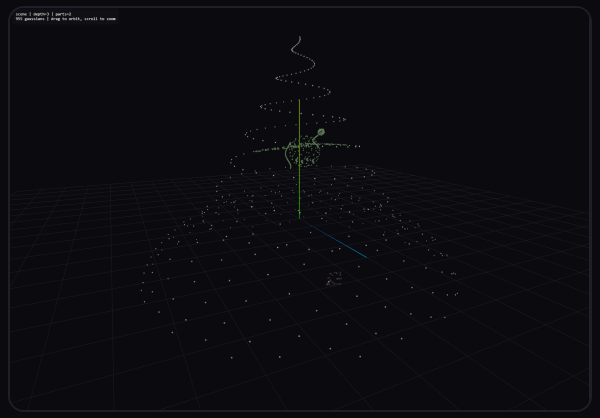
These scenes demonstrate what the pipeline produces with a more capable decomposer. Hand-authored composition trees rendered through the same pipeline, representing the output quality expected from Hertz-scale (1B+) generation.



Castle on a hill
1,124 Gaussians / 4 towers, walls, keep, gate, hill



Medieval village
1,115 Gaussians / houses, market, church, paths



Space station
955 Gaussians / hull, solar arrays, docking bays

1,124
Max Gaussians per scene
(castle decomposition)

100%
Object accuracy
(flat routing, 300 classes)

60fps
Render performance
(WebGL, real-time orbit)

Hand-authored composition trees rendered through the same alpha-compositing pipeline. These demonstrate the target quality for Hertz-scale automated generation.

Structured, queryable, recomposable 3D scenes

Unlike monolithic generation, SGS scenes are structured data. Every part has a name, a parent, a position, and an embedding. This enables applications that require understanding, not just appearance.

Game Asset Pipelines

"Medieval village with a blacksmith" produces an editable scene with labeled buildings, props, and terrain. Designers drag to reposition. Swap components from a library. Export individual parts to game engines. Compose larger worlds from smaller authored scenes.

Architectural Visualization

Building descriptions become 3D walkthroughs with editable floor plans. Change materials per room. Add or remove floors through language commands. Query spatial relationships ("which rooms face south?"). Integrate with BIM metadata.

Film Pre-visualization

Script descriptions become scene layouts. Directors iterate on composition, camera placement, and blocking before any modeling work begins. Each object is individually movable. Lighting can reference objects by name. Export to standard 3D formats for handoff.

Education and Scientific Modeling

"Show me a cell with the mitochondria highlighted" produces a labeled 3D model with queryable parts. Students can hide/show organelles, zoom into sub-structures, and ask spatial questions. Works for molecular biology, astronomy, anatomy, and physics simulations.

Conversational 3D Assistants

Combine blob-based memory with scene generation. "Remember the castle from yesterday, but make the towers taller." The system retrieves prior scene blobs,

Accessibility and Spatial Computing

Because every blob has a semantic label, scenes are inherently accessible. Screen readers can describe spatial relationships. AR/VR systems can attach

Roadmap

From proof-of-concept to production API

Completed

- **Planck 1.0-1.3**
  100M LM, Wikipedia base, blob retrieval (40% recall at 90 turns)
- **Raum 0.1**
  300-class text-to-3D demo, editable DSL, spatial relations
- **Raum 1.3**
  Recursive decomposition pipeline, 5 scenes, WebGL renderer
- **JMLR Submission**
  Alpha-compositing superset proof, Lean 4 verified
- **Planck 1.4**
  Conversation memory blobs, per-turn storage, hybrid retrieval

Next

- **Hertz 1.2** (Q3 2026)
  1B param model, 2048 context, conditional tree generation
- **Raum 2.0** (Q3 2026)
  Hertz-powered decomposer, learned geometry, real-time editing
- **API Beta** (Q4 2026)
  REST API: prompt to structured 3D JSON + render endpoint
- **SDK** (Q1 2027)
  Unity and Unreal Engine plugins, streaming scene updates

Full interactive roadmap visualizer: **pm/index.html** in the repository (reads roadmap.md and renders swimlanes).

RADIANCE LABS

Structured 3D. From Language. At Scale.

We are building the decomposition layer between language models and 3D rendering. Every Gaussian has a name, a parent, and a position in a tree that mirrors how humans think about scenes.

Team

Nikita Gorshkov, Founder
Senior PM, AI/ML infrastructure
Solo researcher, full-stack implementation

Ask

Pre-seed funding to scale the Decomposer LM (1B params), build API infrastructure, and hire a rendering engineer.

github.com/feamando/sgs